HLD Level 2.5

Design a Device Monitoring & Heartbeat System — Complete Interview Question & Answer

Interview question: Design a system that monitors hundreds of thousands of connected devices and determines whether each device is ONLINE, SUSPECT or OFFLINE using periodic heartbeats. The system must scale horizontally, minimize false offline alerts, recover from server failures and handle reconnect storms.

1. Requirements

Functional: receive heartbeats, track last-seen time, expose current device status, detect missing heartbeats, publish status transitions, provide status/history queries, and optionally carry basic health metadata such as version, temperature or error state.

Non-functional: low detection delay, horizontal scalability, high availability, bounded memory, tolerance of network jitter, low false-positive rate, and graceful handling of gateway/monitor failures.

2. Capacity Estimation

```
500,000 devices
heartbeat every 10 sec

Average heartbeats/sec = 500,000 / 10 = 50,000/sec

If heartbeat payload ~= 200 bytes:
~10 MB/sec payload before protocol/transport overhead
```

If every device sends at exactly the same aligned second, the instantaneous rate can be much higher than the average. Device-side jitter should spread heartbeat emission over time.

3. Core Status Model

```
ONLINE -> SUSPECT -> OFFLINE
^ |
+---- heartbeat -----+
```

A binary ONLINE/OFFLINE model can be too sensitive to a single delayed packet. A SUSPECT state provides room for transient network delay or packet loss.

4. Example Timing Policy

```
Expected heartbeat interval: 10 sec
ONLINE: heartbeat recently received
SUSPECT: no heartbeat for >20 sec
OFFLINE: no heartbeat for >30 sec
```

These values are examples, not universal constants. Thresholds should reflect heartbeat interval, network jitter, device behavior and product detection requirements.

5. High-Level Architecture

```
DEVICES
|
v
Load Balancer
|
+--------+--------+
v v v
Gateway1 Gateway2 GatewayN
| | |
+--------+--------+
v
Heartbeat Stream/Broker
|
v
```

```
Monitor Workers
|
+----------+----------+
v v
Latest Status Store Status Events
| |
v v
Query API Alerts/Consumers
```

6. Heartbeat Message

```
{
device_id,
timestamp,
sequence_number,
software_version,
optional health fields
}
```

Keep heartbeat payloads small. Large telemetry should normally use a telemetry path rather than bloating the liveness signal.

7. Gateway Layer

Gateways terminate device connections, authenticate devices, validate heartbeat messages, enforce rate limits and forward valid events. They should remain horizontally scalable.

8. Persistent Connections and C++

For many long-lived TCP connections, a C++ gateway may use event-driven I/O such as epoll rather than one thread per connection.

```
device sockets -> epoll/I-O loop -> parser -> bounded queue -> workers -> broker
```

Use bounded queues between I/O and processing so a slow downstream cannot turn into unlimited memory growth.

9. Heartbeat Processing

When a valid heartbeat arrives, update the device's last_seen only if the event is newer than the currently stored observation. This prevents delayed/out-of-order heartbeats from moving state backward.

```
if incoming.sequence > stored.sequence
or incoming.timestamp > stored.timestamp:
update last_seen/current metadata
```

10. Latest Status Store

```
device_id -> { state, last_seen, last_sequence, gateway/session metadata }
```

A fast key-value/distributed state store is appropriate for current status queries. Long-term status transitions can be stored separately for historical analysis.

11. Why Not Scan Every Device Every Second?

A naive monitor could repeatedly scan all 500,000 devices and compare now-last_seen. That may work at modest scale but wastes repeated work and becomes expensive as fleet size grows.

12. Expiration Scheduling

When a heartbeat arrives, schedule the device's next liveness deadline. Possible implementations include timing wheels, expiration buckets, priority queues or partitioned scheduled scans.

```
heartbeat at 10:00:00
suspect deadline 10:00:20
offline deadline 10:00:30
```

13. Timing Wheel / Buckets

```
10:00:20 bucket -> device A, B, C
10:00:21 bucket -> device D, E
10:00:22 bucket -> device F
```

At each tick, process only devices whose deadlines are due rather than scanning the entire fleet.

14. Stale Timer Problem

Suppose device A sends a heartbeat and gets an offline timer for 10:00:30. It then sends another heartbeat at 10:00:10, creating a newer deadline. The old timer may still fire.

```
Old timer fires -> read current last_seen/version -> verify timer still valid -> only then mark state
```

Never trust a scheduled timeout blindly. Re-check current device state/version when the timeout fires.

15. Partitioning

Partition monitoring work by device_id so ownership and updates scale horizontally.

```
owner_partition = hash(device_id) % N
```

A given device's heartbeat/status transitions should ideally be processed consistently by the same logical partition to reduce races and preserve local ordering.

16. Monitor Worker Failure

If a monitor worker crashes, its partitions must be reassigned. Because current status/deadline information is stored durably or reconstructible from the stream/state store, another worker can resume monitoring.

17. Duplicate Heartbeats

Retries or broker redelivery can create duplicate heartbeat events. Sequence numbers or event IDs allow workers to ignore already-applied events. Status updates should be idempotent.

18. Out-of-Order Heartbeats

```
seq 102 arrives
seq 101 arrives later
```

Do not overwrite last_seen/current metadata with the older sequence. This matters especially after temporary network buffering.

19. False Offline Alerts

Missing one heartbeat should not automatically mean device failure. Causes include packet loss, gateway failover, network jitter, device scheduling pauses or backend delay.

Mitigations include multi-interval thresholds, SUSPECT state, sequence/timestamp validation, ingestion-delay monitoring and confirmation before sending expensive alerts.

20. Backend Delay vs Device Failure

A key challenge is distinguishing a device that stopped sending from a backend that stopped processing. Monitor both device event time and server ingestion/processing time.

```
device_timestamp
gateway_receive_time
monitor_process_time
```

If many devices appear offline simultaneously while broker lag spikes, the monitoring backend may be unhealthy rather than the fleet.

21. Mass Offline / Correlated Failure

If 100,000 devices in one region become offline at once, avoid generating 100,000 independent high-severity alerts without correlation. Aggregate by region/gateway/network/provider and detect common-cause failures.

22. Reconnect Storm

When connectivity returns, devices can reconnect simultaneously. Use exponential backoff plus jitter on clients, connection admission/rate limiting at gateways and sufficient broker buffering.

```
network returns -> randomized reconnect delays -> gateways -> broker -> monitors
```

23. Heartbeat Jitter

Even during normal operation, devices should avoid synchronized heartbeat boundaries. Instead of every device sending exactly at second 0, add a small random offset while preserving the approximate interval.

24. Gateway Failure

When a gateway dies, its connected devices reconnect elsewhere. The monitoring system should allow enough grace time that gateway failover does not immediately classify all those devices as offline.

25. Session Identity

A device can reconnect and create a new session while stale messages from the old session are still in flight. A session/boot ID can help distinguish current and previous connections.

```
device_id + boot_id/session_id + sequence_number
```

26. Split-Brain / Duplicate Connections

If the same device appears through two gateways, define whether multiple sessions are allowed. If not, select the newest authenticated session and reject/ignore stale session updates.

27. Status Transition Events

```
ONLINE -> SUSPECT
SUSPECT -> OFFLINE
OFFLINE -> ONLINE
```

Publish transitions asynchronously so alerting, dashboards, audit history and automation can subscribe without coupling those systems into heartbeat processing.

28. Alerting

Alert only on meaningful transitions and apply deduplication, cooldowns and aggregation. Otherwise intermittent devices can create alert storms.

29. Query APIs

```
GET /devices/{id}/status
GET /devices/{id}/status-history
GET /fleet/status-summary
```

Current state queries hit the latest-status store; historical transitions can use a separate history/time-series store.

30. Reliability Semantics

At-least-once delivery is acceptable when updates are idempotent. Exactly-once end-to-end semantics are usually unnecessary for heartbeat state and significantly complicate the design.

31. Backpressure

If monitor workers fall behind, broker lag grows. If gateways cannot publish fast enough, their bounded queues fill. Apply rate limits and preserve the minimal liveness signal before optional metadata.

32. Graceful Degradation

During severe overload, prioritize core heartbeat fields over nonessential health metadata. The primary purpose of this path is liveness detection.

33. Observability

Track heartbeats/sec, connected devices, gateway connections, invalid heartbeats, duplicate/out-of-order counts, broker lag, monitor processing delay, ONLINE/SUSPECT/OFFLINE counts, state-transition rate, reconnect rate, timer/deadline lag and false-positive/rapid-flap indicators.

> **A sudden rise in OFFLINE count should always be interpreted alongside gateway health, broker lag and processing delay.**

34. C++ State Update Considerations

Use clear ownership for connection/session objects, RAII for sockets and threads, bounded queues, condition variables where worker threads wait, and atomics only for simple independent state. Protect compound device-state transitions with appropriate synchronization or partition ownership.

35. Avoiding Lock Contention

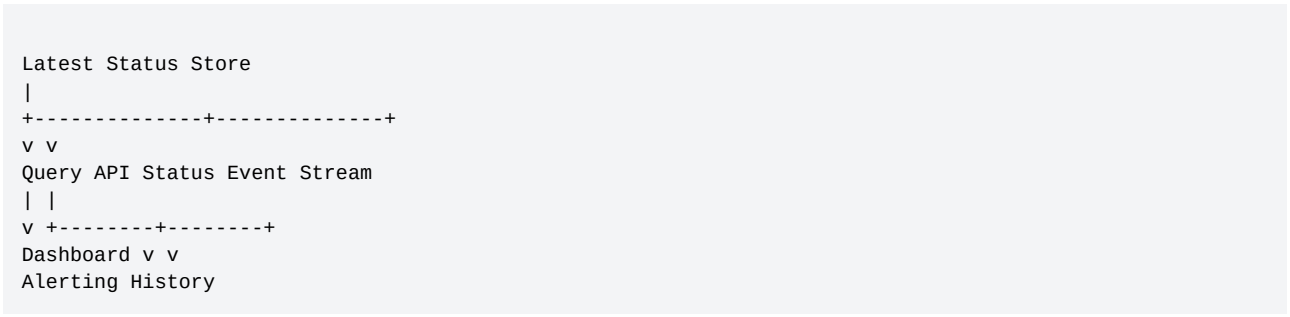
Do not keep one global mutex for all 500,000 devices. Partition/shard device state so workers contend only within a shard, or route each device consistently to a single logical owner.

36. Graceful Shutdown

```
mark gateway/worker not ready
stop accepting new connections/work
drain or checkpoint pending events
flush broker producer/consumer state
stop timer processing safely
join threads
close sockets/resources
```

37. Complete Architecture

```
DEVICES
|
v
Load Balancer
|
+-------------+-------------+
v v v
Gateway 1 Gateway 2 Gateway N
| | |
+-------------+-------------+
v
Heartbeat Broker
|
+-------------+-------------+
v v v
Monitor Part 0 Monitor Part 1 Monitor Part N
| | |
+-------------+-------------+
|
```


Latest Status Store
Query API
Status Event Stream
Dashboard
Alerting History

38. Common Interview Follow-Ups

How quickly do you mark offline? Based on expected interval plus tolerated misses/jitter; often use ONLINE -> SUSPECT -> OFFLINE rather than one missed packet.

Why not scan every device? Deadline scheduling/timing wheels avoid repeated full-fleet scans.

What if an old timer fires? Re-read current last_seen/version before transitioning state.

What if heartbeats arrive out of order? Sequence/timestamp conditional update.

What if a gateway crashes? Devices reconnect with jitter; monitor grace period avoids mass false offline.

What if 100k devices go offline together? Correlate common-cause infrastructure failure and aggregate alerts.

How do monitor workers scale? Partition by device ID and rebalance partitions on worker failure.

How do you avoid a global lock? Sharded state or single logical owner per device partition.

39. Interview-Ready Spoken Answer

"I'd start by estimating the heartbeat rate. With 500,000 devices sending every ten seconds, the average is about 50,000 heartbeats per second. Devices connect through horizontally scalable gateways, which authenticate connections, validate messages and publish small heartbeat events to a partitioned broker. Monitor workers consume events partitioned by device ID and maintain a latest-status record containing state, last-seen time and sequence/session information. I would not scan every device every second; instead I'd schedule liveness deadlines using expiration buckets or a timing wheel. When a deadline fires, the worker re-checks the current last-seen/version so stale timers cannot incorrectly mark an active device offline. To reduce false positives, I'd use multiple missed intervals or an ONLINE-to-SUSPECT-to-OFFLINE transition rather than declaring failure after one delayed heartbeat. Sequence numbers make duplicate and out-of-order events idempotent. The design also handles gateway failure and reconnect storms with grace periods, exponential backoff and jitter. I'd monitor broker lag and processing delay because a backend slowdown can otherwise look like a fleet outage. For C++ gateways, I'd use event-driven socket handling, bounded queues, sharded state and RAII rather than one thread per connection or one global lock."

40. Five Decisions to Remember

1. One missed heartbeat != failure -> ONLINE / SUSPECT / OFFLINE.
2. Do not repeatedly scan the entire fleet -> DEADLINE SCHEDULING / TIMING WHEEL.
3. Old timers and delayed packets exist -> RE-CHECK VERSION / SEQUENCE / LAST_SEEN.
4. Many devices reconnect together -> BACKOFF + JITTER + GATEWAY RATE CONTROL.
5. Many devices appear offline together -> CHECK BACKEND/GATEWAY HEALTH AND CORRELATE COMMON CAUSE.

41. Final Revision Flow

```
Requirements
-> heartbeat rate
-> gateway layer
-> broker
-> partition by device
-> latest state
-> ONLINE/SUSPECT/OFFLINE
-> deadline scheduling
-> stale timer validation
-> duplicate/out-of-order handling
-> gateway failure/reconnect storm
-> alert correlation
-> observability
-> C++ epoll/sharding/RAII
```